XCEL UX DASHBOARD · FOR DESIGNERS

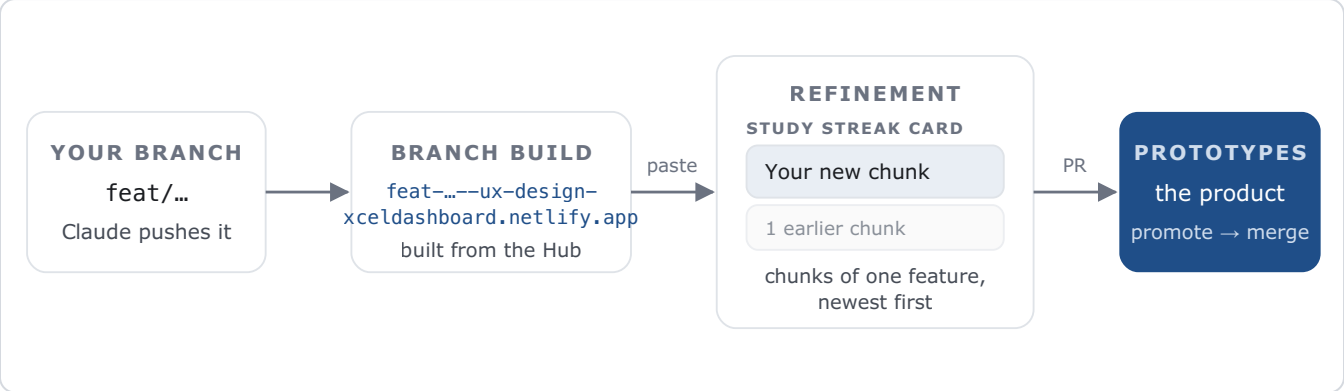
Contributing to the dashboard

How to get your work in front of the team and then in front of stakeholders — from getting the project onto your computer to seeing your change land in Prototypes. Everything happens in the **Claude desktop app**: each step is a sentence you say to Claude. Nothing is typed into a terminal, and github.com is only opened once, to accept the invitation.

Last updated 15 October 2026 · Owner: Jillienne Moreno · Applies to `jillmoreno/xcel-ux-dashboard`

PART 1 · QUICK WALKTHROUGH

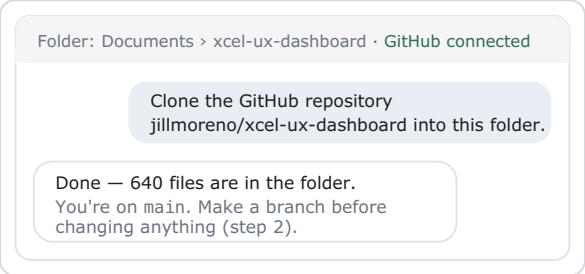
The loop, in one picture



Left to right: you make a branch and Claude pushes your work to it, Netlify builds it at its own address, that address goes into Refinement where it joins the other chunks of the same feature, and a PR plus the promote step lands it in Prototypes. Everyone with the Hub link — the team and stakeholders alike — sees a Refinement row the moment it is added. Everything below is one of those boxes in detail.

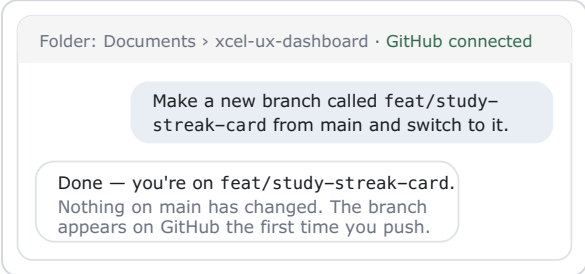
Eight steps

1 Get the project onto your computer



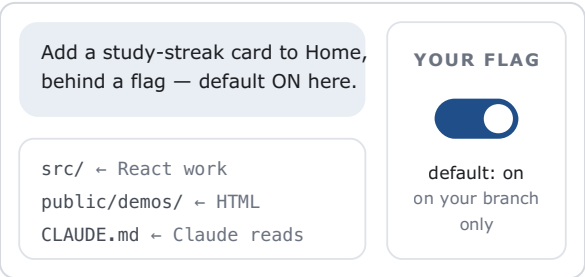
Once. Make an empty folder, open the Claude desktop app, connect that folder, and say *"Clone the GitHub repository jillmoreno/xcel-ux-dashboard into this folder."* Claude brings it down. You do **not** run the site locally — your preview is the branch build in step 5.

2 Make a branch



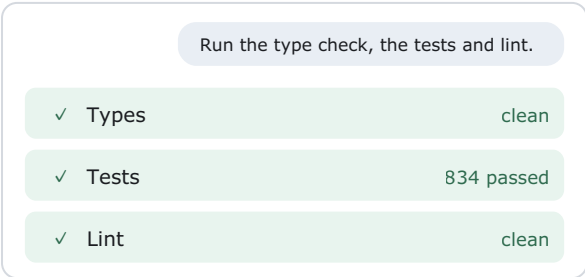
Tell Claude *"Make a new branch called feat/your-thing from main and switch to it."* Name it `feat/` plus two or three words, lowercase, joined with dashes. A branch is your own copy to work in — nothing on it changes what anyone else sees until it is merged.

3 Build it with Claude



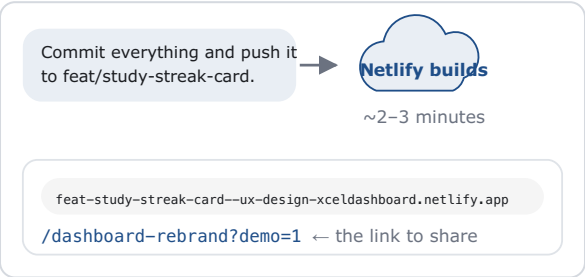
In the Claude desktop app, describe the work. Say which kind it is — a React surface (`src/`) or an HTML exploration (`public/demos/`) — and ask for a feature flag with its default **on** on your branch. Every so often: *"Commit what we have so far."*

4 Ask Claude to run the checks



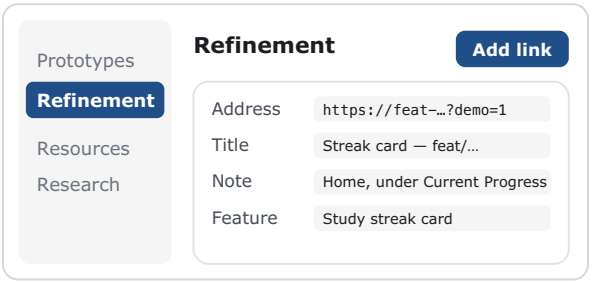
Before every publish, say *"Run the type check, the tests and lint."* Claude runs them and reports. A failing test is usually a recorded decision talking to you — ask Claude what it protects. Touched `public/prototypes/` ? Add *"and the smoke suites."*

5 Publish — that is the deploy



Say *"Commit everything and push it to feat/your-thing."* Claude sends it to GitHub; Netlify builds your branch as its own site. Share `…/dashboard-rebrand?demo=1` so reviewers see your committed defaults. **Every pushed branch gets an address** — anyone with the link and the site password can open it.

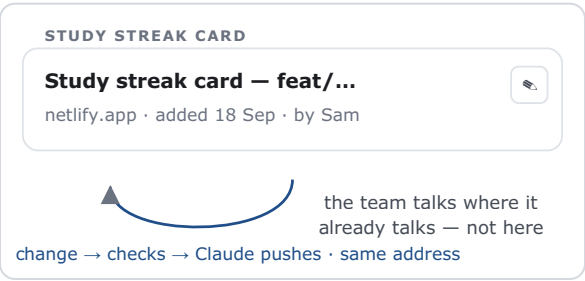
6 Add it to Refinement



Refinement → **Add link**. Paste the address, title it with your branch name, say where to look, and name the **feature** this is a chunk of. Everyone with the Hub link — the team and stakeholders alike — sees it the moment you click Add.

Much faster — it covers steps 4 and 5 as well: in your Claude chat, say `/share`. It runs the checks, commits, pushes, waits for Netlify to build your branch, works out the address, drafts the title and note, asks what feature this is a chunk of, and gives you a link that opens this form already filled in — you check it and click Add. Then it offers to cut your next branch. (`/promote-to-refinement` is the old name and still works.)

7 Discuss and iterate



The team opens the row and talks. You change with Claude, ask for the checks, ask Claude to push again — the address stays the same, so the row stays right. **Edit** the row if its title or note should change.

8 PR → promote → merge → retire



Tell Claude *"Open a pull request from feat/your-thing to main and add Jillienne as reviewer."* She runs `promote-to-prototype` on your branch — each flag your branch changed is kept on or set back to `main`'s value — then merges. The Hub rebuilds. Then say `/retire-from-`

`refinement` to take the row off — it records what was decided before it goes.

Four rules that cover most mistakes

NEVER PUSH TO MAIN

Every push to `main` rebuilds the Hub for everyone. Branch, then PR.

PROMOTION IS A FLAG, NOT A TILE

Prototypes holds one row per product surface — the dashboard, Compass — and a test guards the list. Your work reaches a row by a flag default flipping on `main`.

REVIEW HTML GOES IN PUBLIC/DEMOS/

Both folders work on your branch build. `public/demos/` is for work in review and is meant to be short-lived; `public/prototypes/` is for a document that is becoming a permanent row. Only the second needs a row in the project list.

EVERY BRANCH GETS AN ADDRESS

Anyone with the link and the site password can open it — including work you are not ready to show. Nothing confidential on a branch.

CONTENTS

1. The big picture
2. What you need before you start
3. The eight steps
4. Two kinds of work, two homes
5. Feature flags — the one rule
6. The Refinement form, field by field
7. Things not to do
8. When something looks wrong
9. Glossary

01 The big picture

The dashboard is **one codebase deployed to one website** — the Hub, behind one site password that designers, developers and stakeholders are all given. Every `git push` to `main` rebuilds it. (Until 2026-10-15 there was a second, trimmed site for stakeholders; it is retired.)

GROUP IN THE RAIL	SECTIONS	WHAT THEY ARE
Demo	Prototypes	The product as it stands — one row per surface, every row `main`'s build.
Design & Research	Refinement · Research · Resources · PRDs · Explorations	Four boards authored on the page, then the hand-authored documents — outside products and ideas rebuilt on our tokens.
Dev Handoff	Development · Done · Archive · QA Notes	What engineering is building, what shipped, what was removed, and design-vs-build findings.

Nothing in the rail is locked: the site password is the only door. The old *Designers* group (Design · Exploration behind a second gate) went on 2026-10-15.

Two sections do the work of getting a design in front of people, and they are different acts:

- **Refinement** is the review inbox. It is a list of links you add *on the page* — no code, no commit. A row points at your branch's own build (or an HTML file on it), and it is **one chunk of work**: rows group under the *feature* they are chunks of, newest first, so a run of small branches reads as one piece of work with a history you can still click through. Everyone with the Hub link sees every row.
- **Prototypes** is the product as it stands — the live React build at the flag defaults committed on `main`. Only a merge to `main` changes it, and only Jillienne decides what merges into that baseline.

So the loop is:

branch → `/share` → everyone with the Hub link sees it → discuss & iterate → PR → promote → merge → retire the row → next chunk

The rest of this guide walks each of those, in order, with the exact commands and the exact fields.

02 What you need before you start

- **A GitHub account, with access to the project.** Jillienne invites you; GitHub emails you; click *Accept*. To check it worked, open `github.com/jillmoreno/xcel-ux-dashboard` in your browser — if you see a list of files rather than a 404, you are in.
- **The Claude desktop app, with GitHub connected.** In Claude: *Settings* → *Connectors* → *GitHub* → *Connect*, sign in to GitHub when it asks, and allow it. This is what lets Claude bring the project down to your computer and send your changes back up. Without it Claude can edit files but cannot talk to GitHub.
- **An empty folder on your computer** for the project — for example *Documents › xcel-ux-dashboard*. Claude fills it in step 1.
- **The full site's address.** Ask Jillienne.

> **YOU DO NOT NEED**
>
> Terminal, Node, git, GitHub Desktop, or a Netlify account. Anything that would normally need a typed command, Claude does when you ask it in plain English. You never have to know what the command was.

03 The eight steps

1 Get the project onto your computer

You · once · in the Claude desktop app

What this is. The project lives on GitHub. To work on it, you need a copy in a folder on your own computer that Claude can see and edit. Getting that copy is called *cloning*, and Claude does it for you.

1. **Make an empty folder** somewhere you will find it again — *Documents › xcel-ux-dashboard* is a good choice.
2. **Open the Claude desktop app** and start a new chat. When it asks which folder to work in, choose the folder you just made. (If it does not ask, look for *Connect a folder* or the folder icon near the message box.)
3. **Type this and send it:**
   "Clone the GitHub repository jillmoreno/xcel-ux-dashboard into this folder."
4. **Wait.** Claude uses the GitHub connection to bring everything down. It takes a minute or two. Claude will tell you when it is done.

> **YOU ARE DONE WHEN**
>
> The folder on your computer is no longer empty — you can see `src`, `public`, `CLAUDE.md` and a few dozen other things in it — and Claude has said the clone finished.

You do not run the site on your own computer. Older instructions talked about starting a local server at *localhost:5200*; that needs a terminal, so skip it. Your preview is the branch build Netlify makes in step 5 — a real web address you can open and send to people.

If Claude says it cannot reach GitHub, the connector is not connected — see *What you need*. **If Claude says the folder is not empty**, make a new empty one and connect that instead.

2 Make a branch

You · every time you start a piece of work · one line to Claude

What a branch is, in plain words. The project has one official copy, called `main`. That is what both websites are built from. A *branch* is your own private copy of the project, with a name on it. You make your changes there. Nothing you do on your branch changes `main` — or either website — until Jillienne merges it in step 8. So you cannot break anything by working on a branch. That is the whole point of having one.

1. **Choose a name.** The pattern is `feat/` plus two or three words joined with dashes — for example `feat/study-streak-card`. All lowercase, no spaces. (Use `explore/` for an exploration or `fix/` for a fix; `feat/` is right for most things.)
2. **Tell Claude.** In the Claude desktop app, in your project chat, send:
   "Make a new branch called feat/study-streak-card from main and switch to it."
   Claude makes the branch on your computer and moves your folder onto it, so everything you do from here lands there. If the name breaks the rule, Claude will say so — accept its suggestion.
3. **Nothing to do on GitHub.** The branch exists only on your computer until step 5. The first push creates it on GitHub, and from then on it has its own web address.

> **YOU ARE DONE WHEN**
>
> Claude has confirmed it is on your branch. Ask *"Which branch am I on?"* any time you are unsure — the answer should never be `main` while you are working.

Why the name matters. In step 5 it becomes part of a web address, and the slash turns into a dash: `feat/study-streak-card` is reviewed at `feat-study-streak-card--ux-design-xceldashboard.netlify.app`. Spaces and capital letters would break that, which is why the rule is lowercase and dashes.

If something looks wrong

- **Claude says the name is not valid.** There is probably a space or a capital letter in it. Lowercase, dashes only — take the corrected name Claude offers.
- **Claude says a branch with that name already exists.** Someone (maybe you, earlier) already made it. Either say *"Switch to it and bring it up to date"* to carry on there, or choose a slightly different name.
- **You realise you have been working on `main`.** Do not panic and do not push. Tell Claude: *"I made changes on main by mistake — make a branch called feat/… , move my changes onto it, and leave main untouched."* Claude moves your work across.
- **You want to pick up a branch you started on another computer.** Say *"Switch to the branch feat/… from GitHub and bring it up to date."* Claude fetches it.

3 Do the work with Claude

You · with Claude · in the desktop app

In your project chat, describe what you are building the way you would to a colleague. Claude edits the files in your folder. Three things make this go smoothly:

- **Tell Claude which kind of work it is** — a React surface of the product, or a standalone HTML exploration. Section 04 explains the difference; it decides where the files go.
- **Ask Claude to put new product work behind a feature flag**, and to set that flag's default *on* for your branch. Section 05 explains why. Claude knows the flag catalog (`src/context/FeatureFlagContext.tsx`).
- **Claude reads `CLAUDE.md` on its own** — it is in the folder, and the app picks it up. It is the project's memory: every decision, every trap, every "we tried that and here is why not". A change that contradicts it will be questioned in review.

Save your progress as you go. Every so often, say: *"Commit what we have done so far with the message 'study streak card on Home, behind a flag'."* A commit is a labelled save point. Small ones with plain-English labels are easier for everyone to follow than one big one at the end. Nothing leaves your computer until step 5.

4 Ask Claude to run the checks

You · before every publish

The project has three automatic checks — types, tests and code style. They are the same ones review runs, so green here means no surprises later. Say:

"Run the type check, the tests and lint, and tell me what fails."

Claude runs them in its own workspace (it takes a minute or two — there are about 800 tests) and reports back. If you edited anything under `public/prototypes/` , add *"and the smoke suites"* — those pages have their own checks.

A failing test is usually the project telling you a decision is recorded somewhere — for example, the nav order, or which rows sit on the public front door. Ask Claude what the failing test is protecting; it will read the test's comment and explain. If your change is meant to re-decide that, ask Claude to update the test in the same commit and say so in the message.

5 Publish — that is the deploy

You ask · Claude sends · Netlify builds

What this is. So far your work is only on your computer. *Pushing* sends it up to your branch on GitHub. Claude does it through the GitHub connection. Say:

"Commit everything with the message 'study streak card on Home' and push it to feat/study-streak-card."

Claude confirms. Within a few seconds, Netlify notices the push and starts building your branch as its own website at:

```
https://feat-study-streak-card--ux-design-xceldashboard.netlify.app
```

That is `your-branch-with-dashes` followed by `--ux-design-xceldashboard.netlify.app`. The build takes two to three minutes. Every later push to the same branch updates the same address.

The link to share is the product, in demo mode:

```
https://feat-study-streak-card--ux-design-xceldashboard.netlify.app/dashboard-rebrand?dem
```

`?demo=1` makes the page show your branch's *committed* flag defaults — what you set in step 3 — rather than whatever toggles the person opening it last left in their own browser. Without it, a reviewer may see their own leftover settings and think that is your design.

> **YOU ARE DONE WHEN**
>
> Claude confirms the push went through, and two or three minutes later the address above opens and shows your work. (If you are curious, the branch is now visible at `github.com/jillmoreno/xcel-ux-dashboard/branches` — but you never need to look.)

> **KNOW THIS**
>
> **Every pushed branch gets an address**, and anyone with the link and the site password can open it — including work you are not ready to show yet. That is what makes review possible, and it is why nothing confidential goes on a branch.
>
> A branch address is a copy of the *whole* dashboard, so it carries the team-only sections as well as your work. The house icon in the dark bar is hidden on branch builds for that reason — someone you send a review link to lands on your work, not on the project list.

If Claude says it cannot push, the GitHub connection is missing or has expired — *Settings → Connectors → GitHub*. **If the address 404s after five minutes**, ask Claude *"What is the exact name of*

the branch I pushed?" and check the address uses it, dashes for slashes; then ask Jillienne to confirm branch deploys are on.

6 Add it to Refinement

You · on the Hub

Open the Hub, click **Refinement** in the left nav, then **Add link** (or **Add the first link** if the list is empty). Paste the address from step 5, give it a title, write a one-line note saying where to look, and name the **feature** this is a chunk of. Section 06 has every field.

The row appears immediately for everyone with the Hub link — the team and stakeholders alike. It is saved to a shared store, not to your browser, so it is there the moment you save.

A row is one chunk of work, not one branch you live on. Rows group under their feature, newest chunk first, and when a new chunk carries on from an older one the older one folds under *"1 earlier chunk"* — still there, still clickable. That is the payoff for working small: the feature keeps a history you can open in a review, which one long-running branch cannot give you. If two chunks are two treatments meant to be seen side by side, say so in the form — *Replaces: nothing* — and both stay up.

> **THE SHORT WAY**
>
> In your project chat in the Claude desktop app, say `/share`. It asks you once — checks, commit, push, open the form? — and then does the lot: runs the type check and the tests, commits what is uncommitted, pushes, works out the branch address (dashes and all), waits for Netlify to finish building, drafts the title from your commits, asks you for the one-line note and for the **feature** this is a chunk of, and gives you a link that opens the Hub with the Add form already filled in. You click the link, look, and click **Add link** — nothing is saved without that click. Then it offers to cut your next branch, which is the habit worth having: small chunks, one per branch. (`/promote-to-refinement` is what this was called until 2026-10-15; the old name still reaches it.) Its twin, `promote-to-prototype`, is what Jillienne runs at the other end of the loop (step 8).

7 Discuss, iterate, publish again

You and the team

The team opens your row, looks, and talks — wherever the team talks. You make changes with Claude (step 3), ask for the checks (step 4), and ask Claude to commit and push again (step 5). The address does not change, so the Refinement row is still right. If what the row *is* changes, use **Edit** on the row to update its title or note.

There is deliberately no comment thread on the row. Refinement is a list of things to look at, not a place to record the discussion; that stays where the team already talks.

When the work turns into a different piece of work, start a new branch rather than letting this one grow. Say `/share` again from the new branch and name the same feature: the new chunk goes to the top of the group and this one folds underneath it. `/share` will say something if a branch has got long — more than about ten commits, or older than a week — and that is the moment to split, not later.

8 Open a pull request → promote → merge → retire the row

You open the PR · Jillienne promotes and merges

What a pull request is. It is the formal "please bring my branch into `main`" — a page on GitHub that shows every change, where Jillienne reviews and merges. You open it; she does the rest.

1. **Tell Claude.** *"Open a pull request from feat/study-streak-card to main, titled 'Study streak card', and add Jillienne as reviewer. In the description, put my Refinement note and the branch build address."* Claude opens it through the GitHub connection and gives you the link.
2. **Open the link once** to check the title and description read the way you want. That is the one page on github.com you will see in this whole loop, and you do not have to change anything on it.

If Claude says it cannot open a pull request, the GitHub connection may not have that permission — send Jillienne the branch name and the build address instead, and she will open it.

Jillienne then runs the `promote-to-prototype` skill *on your branch, before merging*. It lists every feature flag your branch changed and asks her, flag by flag, whether that default ships in Prototypes. Approved flags keep your "on"; declined ones are set back to `main`'s value, so the work lands in the sandbox sections of the full site without reaching the public product.

She merges and the Hub rebuilds. The last step is retiring your Refinement row, and the way to do it is `/retire-from-refinement` rather than **Remove**: a row's build lives on a Netlify branch deploy, so deleting the branch deletes the only viewable copy of the work, and “we tried this and decided not to” otherwise leaves no trace anywhere. The skill makes the keep-or-delete-the-branch call explicit and writes the Archive row before the Refinement row goes.

> **DONE LOOKS LIKE**
>
> Your flag's default is on in `main`, the Prototypes section shows your work at `/dashboard-rebrand?demo=1`, and Refinement no longer lists this chunk — though the feature's earlier chunks are still there to scroll back through, if any are.

04 Two kinds of work, two homes

Where files go depends on what you are making, and picking wrong is the most common mistake in this repo.

YOU ARE MAKING	IT LIVES IN	REVIEWERS OPEN	PROMOTED BY
A React surface of the product	`src/` — a page in the shell, a card, a panel, a variant of an existing block. Behind a feature flag.	`<branch>--…netlify.app/dashboard-rebrand?demo=1`	Flipping the flag's default on `main` (the promote skill).
A standalone HTML exploration	`public/demos/<your-file>.html` — a hand-authored page answering a brief, not wired into the app.	`<branch>--…netlify.app/demos/<your-file>.html`	Moving it to `public/prototypes/` with a row in `prototypeFeatures.ts` if it is a finished exploration, or rebuilding it in `src/` if it became a product surface.

> **WHY `PUBLIC/DEMOS/` AND NOT `PUBLIC/PROTOTYPES/`**
>
> `public/prototypes/` holds the finished standalone explorations — each one has a row in the feature list and its own smoke assertions, and the user-test build 404s the whole folder at the edge. `public/demos/` is the folder for HTML in review, and it is deliberately left open everywhere. Nothing should live there long, and nothing there gets a row in the feature list — the Refinement panel is its listing.

05 Feature flags — the one rule

Every change to the product starts **invisible to Prototypes**: behind a flag whose default on `main` is off (or set to the existing variant). That is what lets many designers work on the same product without anyone's half-finished idea appearing in front of stakeholders.

On **your branch**, you set that flag's default *on*. That is what makes your branch build show your work at `?demo=1`. It also means the promote skill can see, mechanically, exactly which defaults your branch changed — the list it asks Jillienne about in step 8 is the diff of the flag catalog between your branch and `main`.

- Flags live in `src/context/FeatureFlagContext.tsx`, in `FEATURE_FLAGS`. Ask Claude to add yours there and to add it to the `/dashboard-rebrand` panel scope so reviewers can toggle it.

- Reviewers can flip flags live on your build: hover the prototype bar and find the robot icon, which opens the flag sheet. In demo mode those flips are temporary and reset on exit.
- Do not promote by adding a row to Prototypes. That section holds one row per product surface — all the same build on `main` — and a test compares it in both directions, so an unexpected row fails the suite.

06 The Refinement form, field by field

This is what **Add link** opens on the Hub.

REFINEMENT · ADD LINK

Address	The full `https://…` link. Required. Only `http` and `https` addresses are accepted — anything else is refused, on purpose, because every address here is rendered as a real link for everyone.
Title	What it is, in a few words. Required. Include the branch name so the row can be matched to a PR later: *"Study streak card — feat/study-streak-card"*.
Note	Optional, but use it. Say where to look and what question you want answered: *"Home, under Current Progress. Is the streak motivating or nagging?"*
Added by	Your name, from the roster. Optional; remembered in your browser so you choose it once. It is a label, not a login — nothing checks it — and it is what the filter pills along the top of the list filter on.
Product	Atlas · Compass · Both. Which product this work is about, which is also which Prototypes row it is heading for. `/share` reads it off your branch, so it is usually already right.
Feature	What this chunk is *part of*. Optional, free text, and the field the list groups on. Start typing and the features already on the board are offered — take one if it fits, because *"Streak card"* and *"streak card"* are the same group but *"Streaks"* is a different one. There is no fixed list to pick from on purpose: a new feature should never wait on someone else adding a value.
Replaces	Only appears once the feature you typed already has chunks on the board. It defaults to **your own newest chunk** of that feature, because that is usually what this is — the next piece of the same work. The first option is *"Nothing — these sit side by side"*, and that is a real answer: two treatments of one surface, both up at once, is exactly what Refinement is for.

On the list, rows sit under their feature, newest chunk first; a chunk something else replaced folds under **"N earlier chunks"**, one click from being open again. A row with no feature has no heading and sits

where its date puts it, exactly as every row did before this existed — there was no backfill, so the board is mixed for a while.

There is no *Type* field here, unlike Resources: every Refinement row is the same kind of thing — one chunk of work in review. There is no *public* toggle either. There was one until 2026-10-15, when the separate stakeholder site was retired; a row is now visible to everyone with the Hub link the moment it is added.

07 Things not to do

- **Don't push to `main`.** Every push to `main` rebuilds the Hub, which stakeholders are looking at. Always be on a branch before asking Claude to push — step 2 — and get to `main` only through a pull request.
- **Don't add a row to Prototypes** or change a row's `category` to `'prototype'`. Promotion is a flag default, never a tile move.
- **Don't put review HTML in `public/prototypes/`.** That folder is the finished standalone explorations, each with a row in the feature list and its own smoke assertions. Work in review goes in `public/demos/`, which exists for exactly that and gets no row.
- **Don't change a flag's default on `main` yourself.** That is the promote step, and it is Jillienne's call.
- **Send your branch build, never the main dashboard.** Your branch address shows your work; the main address shows what is already decided, and everything else the team is doing.
- **Don't invent data.** A number, a URL, a lesson count or a fee that XCEL has not published is flagged as `INVENTED` in the code and named as such on screen. `CLAUDE.md` records why, several times over.
- **Don't draft the Quick Summary** on a dev-handoff component — ask Jillienne for it. The `dev-handoff-notes` skill says the same.

08 When something looks wrong

SYMPTOM	LIKELY CAUSE	TRY
Branch address 404s	Netlify hasn't finished the build, or branch deploys were switched off.	Wait three minutes and rel fails, check the branch nar slashes turned to dashes; Jillienne to confirm branch on.
Branch shows old main, not your work	Your flag default is off on the branch, or you opened the page without `?demo=1`.	Add `?demo=1`. Then ask *my flag's default on for thi*
The demo looks different on one laptop only	That browser holds a saved "Set as default" flag snapshot, which wins over the committed default per key.	In that browser's DevTools Application → Local Stora `cgp.featureFlags.cust`
Claude says it cannot reach GitHub, clone, or push	The GitHub connector is not connected, or its sign-in expired.	Claude desktop → *Setting Connectors* → *GitHub* → Reconnect), sign in, allow again.
Row shows "not a linkable address"	The address	

SYMPTOM	LIKELY CAUSE	TRY
	wasn't `http(s)`, or was saved by an older version.	Edit the row and paste the `https://` address.
Stakeholder can't see your row	They don't have the Hub link or the site password — there has been no per-row visibility switch since 2026-10-15.	Send them the Hub link. E visible to everyone who ca site.
Your chunk isn't under the feature you expected	The feature was typed slightly differently — grouping ignores case and spacing, but not different words.	Edit the row and pick the f the suggestions rather tha The group takes its spellin newest chunk.
An older chunk vanished from the board	It didn't — a newer chunk replaced it, so it folded under **"N earlier chunks"** at the top of the group.	Click that. If it should not h replaced, edit the newer ro *Replaces* to *Nothing*.

SYMPTOM	LIKELY CAUSE	TRY
Tests fail on something you didn't touch	A decision is pinned — nav order, section contents, a colour that must keep failing.	Ask Claude what the failin protects — it reads the tes Either your change re-dec (Claude updates the test a in the commit) or it found a regression.
Claude cannot find your project files	The chat is not connected to the project folder, or it is connected to the wrong one.	Start a new chat and choo *ux-dashboard* folder when Claude should be able to l `CLAUDE.md` and `src`.
Claude says something is locked or "another process is running"	An earlier action did not finish cleanly.	Say: *"Clear the git lock an* Claude knows what to do.

09 Glossary

Repo	The project's code, in git. `jillmoreno/xcel-ux-dashboard` on GitHub.
Clone	Bring the project down into a folder on your computer, once. Claude does it when you ask.
Commit	A labelled save point. "Commit with the message …" — Claude does it; nothing leaves your computer yet.
Branch	Your own line of work inside the repo. Nothing on it is public until merged — but its *build* has a public address.
Push	Send your commits up to your branch on GitHub. Claude does it through the GitHub connector. Netlify builds every pushed branch.
GitHub connector	The link between the Claude desktop app and your GitHub account (*Settings → Connectors*). It is what lets Claude clone, push and open pull requests for you.
Pull request (PR)	The GitHub page that asks for your branch to be merged into `main`. You ask Claude to open it; Jillienne reviews, promotes and merges.
Branch deploy	The website Netlify builds for one branch: `<branch>--ux-design-xceldashboard.netlify.app`.
`?demo=1`	Demo mode: render the committed flag defaults, ignore the viewer's own saved toggles, don't persist flips.
Feature flag	A named switch in `FEATURE_FLAGS`. Off on `main` until promoted; on on your branch so your build shows it.
Prototypes	The section holding the live product build at `main`'s flag defaults. One row per product surface, all the same build. Changed only by merging.
Refinement	The review inbox: links added on the page, each one a chunk of work, grouped under the feature it belongs to. Its internal id is `demo` — you will see that in URLs (`?section=demo`) and code.
Resources	Everything else that lives elsewhere: briefs, boards, Figma files. Also added on the page.
Promote	Two skills bracket the loop. `/share` (yours, on the branch — called `promote-to-refinement` until 2026-10-15) checks, commits, pushes and opens the Refinement form prefilled with your branch address. `promote-to-prototype` (Jillienne's, on the PR) decides per flag whether its "on" default ships in Prototypes.

The Hub	The one site, behind one password, that designers, developers and stakeholders are all given. Until 2026-10-15 there was a second, trimmed site for stakeholders; you may still see “full site” in older notes, and it means this one. A separate user-test build still exists and is not it.

Part of the XCEL UX Dashboard. The reasoning behind every decision described here is in the repo's `CLAUDE.md`; the one-time site setup is in `README.md`. Stakeholders get a shorter companion page, How to read this dashboard.